KIUWAN REPORT

VULNERABILITIES REPORT

Application: SUGIPQ-V2_Pruebas
Analysis Label:
Analysis Date: 2026/01/28 16:12
Report Date: 2026/01/28 09:18

APPLICATION & ANALYSIS INFORMATION

This software has been scanned and vulnerabilities checked for based on the following standards: OWASP, CWE, SANS 25, PCI-DSS, HIPAA, WASC, MISRA-C, BIZEC, ISO 25000, ISO 9126, CERT-C, CERT-J. Kiuwan has analyzed for you a set of source files. In order to put this analysis and your results in context, here are some statistics:

Notes:

- unparsed files are the source code files of your application that kiuwan engine couldn't read during the analysis process.
- unrecognized files are the ones that were in your analyzed path but they aren't source code or they contains code in languages that Kiuwan doesn't support yet.

Analysis Info

label	no label
date	2026/01/28 09:09
ordered by	Eric Torres
encoding	UTF-8
analyzed path	-
languages found	html, javascript, python
analyzed files	107
unparsed files	7
unrecognized files	-

Application Info

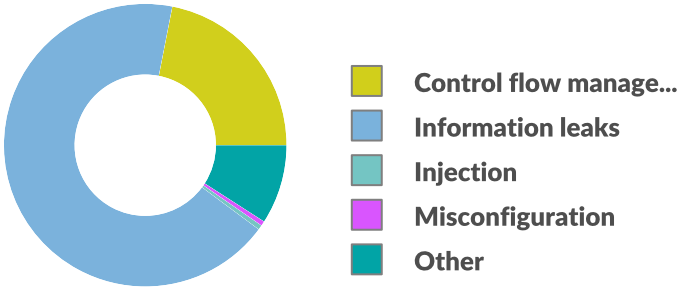
name	SUGIPQ-V2_Pruebas
Bussiness value	critical
Times analyzed	4

Model Info

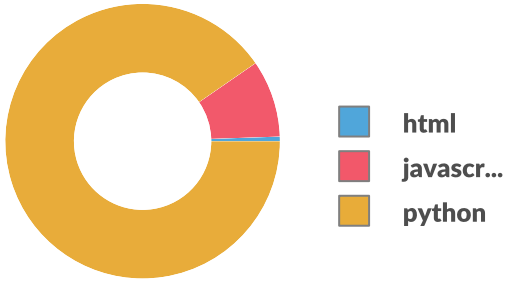
model name	CQM
model version	q13605
engine version	master.p703.q13605.a1914.i670
active rules	280
active metrics	78

VULNERABILITIES DISTRIBUTION

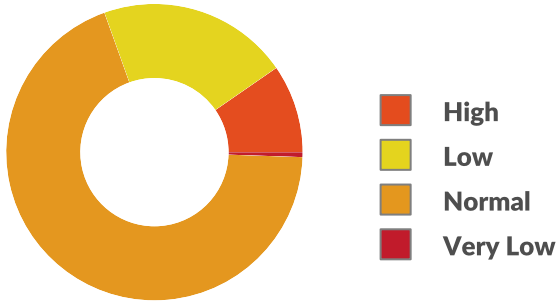
BY TYPE



BY LANGUAGE



BY PRIORITY



VIOLATED RULES

6

SECURITY RATING

VULNERABILITIES

187

VERY HIGH

0

VULNERABILITIES

FILES	#VULNERABILITY	RULE	CHARACTERISTIC
8	17	Avoid unused local variable	Maintainability

PRIORITY	CWE	VULNERABILITY TYPE	LANGUAGE
	563	Other	javascript

This rule identifies any unused local variable in a function.

1 Defects in file templates/aprobadores/listar.html

```
171 tooltipList = tooltipTriggerList.map(function (tooltipTriggerEl) {
      return new bootstrap.Tooltip(tooltipTriggerEl);
    })
```

1 Defects in file templates/auth/login.html

```
163 newState = isHidden ? 'visible' : 'oculta'
```

5 Defects in file templates/dashboard.html

```
548 dia = ahora.getDate().toString().padStart(2, '0')
549 mes = (ahora.getMonth() + 1).toString().padStart(2, '0')
```

```
550 año = ahora.getFullYear()
556 hora = ahora.getHours().toString().padStart(2, '0')
557 minutos = ahora.getMinutes().toString().padStart(2, '0')
```

1 Defects in file templates/inventario_corporativo/devolver.html

```
229 producto = document.getElementById('preview-producto').textContent
```

4 Defects in file templates/prestamos/listar.html

```
820 totalPrestamos = document.getElementById('total-prestamos').textContent
821 pendientes = document.getElementById('pendientes').textContent
822 aprobados = document.getElementById('aprobados').textContent
823 devueltos = document.getElementById('devueltos').textContent
```

2 Defects in file templates/reportes/novedades.html

```
462 fecha = new Date().toISOString().split('T')[0]
488 fecha = new Date().toLocaleDateString('es-ES', {
      day: '2-digit',
      month: '2-digit',
      year: 'numeric'
```

```
})
```

2 Defects in file templates/reportes/prestamos.html

```
309 nombreOficina = document.getElementById('filtro_oficina').options[document.getElementById('filtro_oficina').selectedIndex].text
332 nombreOficina = document.getElementById('filtro_oficina').options[document.getElementById('filtro_oficina').selectedIndex].text
```

1 Defects in file templates/usuarios/gestionar.html

```
583 usuario = response.usuario
```

FILES	#VULNERABILITY	RULE	CHARACTERISTIC
1	1	Unvalidated untrusted input in log	Security

PRIORITY	CWE	VULNERABILITY TYPE	LANGUAGE
	117	Injection	python

When the application writes untrusted input to a log destination, using any Python logging API, a "log forging" attack could be launched to inject false log entries, corrupt log entries, or even exploit a vulnerability in an external "log viewer" application.

Corrupted log entries could hinder detection for malicious activity, or implicate another party in the commission of a malicious act.

1 Sink in file blueprints/oficinas.py

```
94 logger.info("  Oficina creada: %s (ID: %s)", sanitizar_log_text(nombre), sanitizar_log_text("%s" % oficina_id))
```

Propagation path #1

Source blueprints/oficinas.py

```
75 | nombre = request.form.get('nombre')
```

FILES	#VULNERABILITY	RULE	CHARACTERISTIC
5	88	Avoid sensitive information exposure through error messages	Security

PRIORITY	CWE	VULNERABILITY TYPE	LANGUAGE
	209	Information leaks	python

When an error message is generated, care should be taken to ensure that it does not contain sensitive information about the environment, users or any other information that may be considered sensitive.

Such information may be valuable itself or may be useful for further attacks with a higher impact.

Attention should be paid to information explicitly included into error messages, and also to certain functions that may produce an error that unexpectedly shows sensitive information.

Rule shows violation when some sensitive error information, or with excessive technical detail, is shown to the application end-user.

---

43 Sink in file blueprints/confirmacion_asignaciones.py

```
115 | logger.error(
        " UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
        template_name, 'Error interno'
    )
```

Propagation path #1

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #2

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
```

```
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #3

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
```

```
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #4

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
```

```
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #5

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
```

```
    logger.error(
        "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
        template_name, 'Error interno'
    )

    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #6

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
```

```
    )

    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #7

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
```

```
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #8

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
```

```
        return render_template(template_name, **context)
    except Exception as e2:
        logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                     template_name, 'Error inte
```

Propagation path #9

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
```

```
                    template_name, 'Error inte
```

Propagation path #10

Source blueprints/confirmacion_asignaciones.py

```
105 def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte

122 logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
```

Propagation path #1

Source blueprints/confirmacion_asignaciones.py

```
114 | e
```

Propagation path #2

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
```

```
                    template_name, 'Error inte
```

Propagation path #3

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #4

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #5

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
```

```
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #6

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
```

```
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #7

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)
```

```
except UnicodeDecodeError as e:
    logger.error(
        "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
        template_name, 'Error interno'
    )

    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #8

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
```

```
        "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
        template_name, 'Error interno'
    )

    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #9

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )
```

```
    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #10

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
```

```
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #11

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
```

```
    except Exception as e2:
        logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                     template_name, 'Error inte
```

```
126 logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                     template_name, 'Error interno')
```

Propagation path #1

Source blueprints/confirmacion_asignaciones.py

```
114 e
```

Propagation path #2

Source blueprints/confirmacion_asignaciones.py

```
105 def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
```

```
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #3

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
```

```
        return render_template(template_name, **context)
    except Exception as e2:
        logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                     template_name, 'Error inte
```

Propagation path #4

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
```

```
                    template_name, 'Error inte
```

Propagation path #5

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                            template_name, 'Error inte
```

Propagation path #6

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #7

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
```

```
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #8

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
```

```
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #9

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)
```

```
except UnicodeDecodeError as e:
    logger.error(
        "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
        template_name, 'Error interno'
    )

    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #10

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
```

```
        "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
        template_name, 'Error interno'
    )

    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #11

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )
```

```
    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

129 logger.error(" No se pudo reparar encoding para '%s': %s", template_name, msg)

Propagation path #1

Source blueprints/confirmacion_asignaciones.py

114 e

Propagation path #2

Source blueprints/confirmacion_asignaciones.py

105 def safe_render_template(template_name: str, **context: Any):

```
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
```

```
        "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
        template_name, 'Error interno'
    )

    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #3

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )
```

```
    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #4

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
```

```
        logger.info("   Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("   Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #5

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "   UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("   Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
```

```
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #6

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #7

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #8

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #9

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
```

```
Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
ya se realizó en BD.
"""
try:
    return render_template(template_name, **context)

except UnicodeDecodeError as e:
    logger.error(
        "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
        template_name, 'Error interno'
    )

    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

Propagation path #10

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
```

```
    """
    try:
        return render_template(template_name, **context)

    except UnicodeDecodeError as e:
        logger.error(
            "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
            template_name, 'Error interno'
        )

        ok, msg = _ensure_template_utf8(template_name)
        if ok:
            logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
            try:
                return render_template(template_name, **context)
            except Exception as e2:
                logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                             template_name, 'Error inte
```

Propagation path #11

Source blueprints/confirmacion_asignaciones.py

```
105 | def safe_render_template(template_name: str, **context: Any):
    """
    Renderiza una plantilla y, si falla por UnicodeDecodeError, intenta convertirla a UTF-8
    y reintenta una vez. Esto evita que el usuario vea 'error' cuando la confirmación
    ya se realizó en BD.
    """
    try:
        return render_template(template_name, **context)
```

```
except UnicodeDecodeError as e:
    logger.error(
        "  UnicodeDecodeError al renderizar plantilla '%s': %s. Intentando convertir a UTF-8 y reintentar...",
        template_name, 'Error interno'
    )

    ok, msg = _ensure_template_utf8(template_name)
    if ok:
        logger.info("  Reparación de encoding aplicada para '%s': %s", template_name, msg)
        try:
            return render_template(template_name, **context)
        except Exception as e2:
            logger.error("  Falló el reintento de render_template('%s') después de convertir a UTF-8: %s",
                         template_name, 'Error inte
```

10 Sink in file blueprints/materiales.py

```
110 | logger.error(sanitizar_log_text(f"    Error procesando estadísticas del material {material.get('id', 'desconocido')}: {e}"))
```

Propagation path #1

Source blueprints/materiales.py

```
109 | e
```

```
115 | logger.error(sanitizar_log_text(f"    Error general en cálculo de estadísticas: {e}"))
```

Propagation path #1

Source blueprints/materiales.py

```
114   e
```

```
225 logger.error(sanitizar_log_text(f"  Error al crear materiales: {e}"))
```

Propagation path #1

Source blueprints/materiales.py

```
224 e
```

```
227 logger.debug(sanitizar_log_text('Traceback omitido por seguridad'))
```

Propagation path #1

Source blueprints/materiales.py

```
224 e
```

```
323 logger.error(sanitizar_log_text(f"  Error editando material: {e}"))
```

Propagation path #1

Source blueprints/materiales.py

```
322 e
```

```
325 logger.debug(sanitizar_log_text('Traceback omitido por seguridad'))
```

Propagation path #1

Source blueprints/materiales.py

```
322 e
```

```
361 logger.error(sanitizar_log_text(f"  Error eliminando material: {e}"))
```

Propagation path #1

Source blueprints/materiales.py

360 | e

50 | logger.error(sanitizar_log_text(f" Error obteniendo estadísticas para material {material_id}: {e}"))

Propagation path #1

Source blueprints/materiales.py

49 | e

81 | logger.error(sanitizar_log_text(f" Error obteniendo materiales: {e}"))

Propagation path #1

Source blueprints/materiales.py

80 | e

83 | logger.debug(sanitizar_log_text('Traceback omitido por seguridad'))

Propagation path #1

Source blueprints/materiales.py

80 | e

---

1 Sink in file config/permissions.py

137 | logger.error('Error importando configuraciones: [error](%s)', sanitizar_log_text(type(e).__name__))

Propagation path #1

Source config/permissions.py

135 e

8 Sink in file services/auth_service.py

58 logger.warning("Falló validación de directorio para %s (id=%s)", sanitizar_log_text(sanitizar_username(username)), sanitizar_log_text(_new_error_id()))

Propagation path #1

Source services/auth_service.py

52 e

60 logger.info("Intentando validación local para %s", sanitizar_log_text(sanitizar_username(username)))

Propagation path #1

Source services/auth_service.py

57 ldap_error

66 logger.info("Validación local exitosa para %s", sanitizar_log_text(sanitizar_username(username)))

Propagation path #1

Source services/auth_service.py

57 ldap_error

71 logger.warning("Flask-Login no está configurado o ocurrió un error interno (id=%s)", sanitizar_log_text(_new_error_id()))

Propagation path #1

Source services/auth_service.py

57 ldap_error

76 logger.error("Error interno en validación local (id=%s)", sanitizar_log_text(_new_error_id()))

Propagation path #1

Source services/auth_service.py

57 ldap_error

Propagation path #2

Source services/auth_service.py

70 e

78 logger.warning("Credenciales inválidas para %s", sanitizar_log_text(sanitizar_username(username)))

Propagation path #1

Source services/auth_service.py

57 ldap_error

Propagation path #2

Source services/auth_service.py

75 db_error

---

26 Sink in file solicitudes.py

247 logger.error('Error obteniendo email: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

246 e

291 logger.error('Error obteniendo info solicitud: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

290 e

356 logger.error('Error al listar solicitudes: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

355 e

398 logger.error('Error enviando notificación de solicitud creada: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

397 e

409 logger.error('Error al crear solicitud: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

408 e

434 logger.error('Error enviando notificación de aprobación: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

433 e

442 logger.error('Error al aprobar solicitud %s: %s',
sanitizar_log_text(solicitud_id),
sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

441 e

477 logger.error('Error enviando notificación de aprobación parcial: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

476 e

483 logger.error('Error al aprobar parcial solicitud %s: %s',
sanitizar_log_text(solicitud_id),
sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

482 e

514 logger.error('Error enviando notificación de rechazo: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

513 e

520 logger.error('Error al rechazar solicitud %s: %s',
sanitizar_log_text(solicitud_id),
sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

519 e

570 logger.error('Error al solicitar devolución %s: %s',
sanitizar_log_text(solicitud_id),
sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

569 e

602 logger.error('Error al aprobar devolución: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

601 e

632 logger.error('Error al rechazar devolución: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

631 e

646 logger.error('Error obteniendo devolución pendiente %s: %s',
sanitizar_log_text(solicitud_id),
sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

645 e

715 logger.error('Error enviando notificación de novedad: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

714 e

722 logger.error('Error al registrar novedad: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

721 e

780 logger.error('Error enviando notificación de gestión novedad: %s', sanitizar_log_text(e))

Propagation path #1

Source solicitudes.py

779 e

```
788 logger.error('Error en gestión de novedad: %s', sanitizar_log_text(e))
```

Propagation path #1

Source solicitudes.py

```
787 e
```

```
812 logger.error('Error al listar novedades: %s', sanitizar_log_text(e))
```

Propagation path #1

Source solicitudes.py

```
811 e
```

```
826 logger.error('Error al obtener novedades pendientes: %s', sanitizar_log_text(e))
```

Propagation path #1

Source solicitudes.py

```
825 e
```

```
840 logger.error('Error obteniendo novedad para solicitud %s: %s',
            sanitizar_log_text(solicitud_id),
            sanitizar_log_text(e))
```

Propagation path #1

Source solicitudes.py

```
839 e
```

```
862 logger.error('Error obteniendo info devolución %s: %s',
            sanitizar_log_text(solicitud_id),
```

```
            sanitizar_log_text(e))
```

Propagation path #1

Source solicitudes.py

```
861 | e
```

```
880 | logger.error('Error obteniendo detalle de solicitud %s: %s',
            sanitizar_log_text(solicitud_id),
            sanitizar_log_text(e))
```

Propagation path #1

Source solicitudes.py

```
879 | e
```

```
894 | logger.error('Error obteniendo estadísticas: %s', sanitizar_log_text(e))
```

Propagation path #1

Source solicitudes.py

```
893 | e
```

```
923 | logger.error('Error actualizando novedad: %s', sanitizar_log_text(e))
```

Propagation path #1

Source solicitudes.py

```
922 | e
```

FILES	#VULNERABILITY	RULE	CHARACTERISTIC
7	41	Execution After Redirect (EAR)	Security

PRIORITY	CWE	VULNERABILITY TYPE	LANGUAGE
	698	Control flow management	python

The web application sends a forward / redirect to another location, but instead of exiting, it executes additional code.

If the redirect is a client redirect (typically, setting the Location HTTP header), the attacker may ignore such redirect and process the message body, potentially accessing unintended contents rendered to the HTTP response after the redirect.

When the application developer does not terminate the response, but perform additional operations assuming that the redirect call will not end the flow, then an unauthorized user may be able to access unintended contents.

The rule looks for statements following the forward / redirect call, and reports a defect when a non allowed statement follows that call. A few kind of statements, like a return, a break (in a switch case), a logging call, or a counter increment, might be allowed.

---

4 Defects in file blueprints/aprobacion.py

```
26 return redirect('/solicitudes')
53 return redirect('/solicitudes')
60 return redirect('/solicitudes')
88 return redirect('/solicitudes')
```

---

6 Defects in file blueprints/inventario_corporativo.py

```
330 return redirect('/inventario-corporativo')
```

385 return redirect(f'/inventario-corporativo/{producto_id}')

456 return redirect(request.url)

460 return redirect(request.url)

523 return redirect(f'/inventario-corporativo/{producto_id}')

536 return redirect(f'/inventario-corporativo/{producto_id}')

7 Defects in file blueprints/materiales.py

222 return redirect('/materiales')

263 return redirect('/materiales')

267 return redirect('/materiales')

320 return redirect('/materiales')

345 return redirect('/materiales')

349 return redirect('/materiales')

358 return redirect('/materiales')

5 Defects in file blueprints/oficinas.py

128 return redirect(f'/oficinas/editar/{oficina_id}')

145 return redirect('/oficinas')

156 return redirect('/oficinas')

221 return redirect('/oficinas')

96 return redirect('/oficinas')

---

11 Defects in file blueprints/reportes.py

1,141 return redirect('/reportes')

1,177 return redirect('/reportes')

```
1,340 return redirect('/reportes')
1,378 return redirect('/reportes')
1,513 return redirect('/reportes')
1,549 return redirect('/reportes')
1,692 return redirect('/reportes/materiales')
1,698 return redirect('/reportes/materiales')
1,925 return redirect('/reportes')
2,065 return redirect('/reportes')
2,202 return redirect(f"/reportes/oficinas/{oficina_id_val}/inventario/excel?" + urlencode(params))
```

4 Defects in file blueprints/solicitudes.py

```
274 return redirect('/solicitudes/crear')
279 return redirect('/solicitudes/crear')
291 return redirect('/solicitudes')
294 return redirect('/solicitudes/crear')
```

4 Defects in file solicitudes.py

```
373 return redirect('/solicitudes/crear')
379 return redirect('/solicitudes/crear')
401 return redirect('/solicitudes')
404 return redirect('/solicitudes/crear')
```

FILES	#VULNERABILITY	RULE	CHARACTERISTIC
8	39	Avoid exposing sensitive information through log	Security

PRIORITY	CWE	VULNERABILITY TYPE	LANGUAGE
	532	Information leaks	python

Logs can be very helpful at development stages but they should be used on later stages because they can show rewarding information.

When a log shows sensitive information, it can create a security leak, making useless any other system protection mechanisms. Also sensible data can have many legal implications due to the established limitations about personal information storage.

This rule reports violation when the log shows information that could be considered sensitive.

---

1 Sink in file blueprints/aprobadores.py

```
22 logger.warning("Usuario %s sin permisos para ver aprobadores", sanitizar_log_text(session.get("usuario_id")))
```

Propagation path #1

Source blueprints/aprobadores.py

```
session
```

1 Sink in file blueprints/auth.py

```
30 logger.debug("Verificación de autenticación: %s", sanitizar_log_text(is_authenticated))
```

Propagation path #1

Source blueprints/auth.py

```
30 logger.debug("Verificación de autenticación: %s", sanitizar_log_text(is_authenticated))
```

1 Sink in file blueprints/materiales.py

```
59 logger.warning(sanitizar_log_text(f"⚠ Acceso denegado a /materiales - Usuario: {session.get('usuario_nombre')}"))
```

Propagation path #1

Source blueprints/materiales.py

```
11 session
```

1 Sink in file blueprints/oficinas.py

```
27 logger.warning("    Usuario %s sin permisos para ver oficinas", sanitizar_log_text(sanitizar_username(session.get("usuario"))))
```

Propagation path #1

Source blueprints/oficinas.py

```
3 session
```

7 Sink in file config/permissions.py

```
100 logger.debug('Rol encontrado por comparación plana: %s', sanitizar_log_text(key))
```

Propagation path #1

Source config/permissions.py

```
100 logger.debug('Rol encontrado por comparación plana: %s', sanitizar_log_text(key))
```

```
160 logger.debug('Permisos obtenidos para usuario: role_key=%s, office_key=%s', sanitizar_log_text(safe_role_key), sanitizar_log_text(safe_office_key))
```

Propagation path #1

Source config/permissions.py

```
158 safe_role_key = sanitizar_log_text(role_key)
```

```
307 logger.debug('Filtro de oficina: %s', sanitizar_log_text(safe_office_key))
```

Propagation path #1

Source config/permissions.py

```
306 safe_office_key = sanitizar_log_text(office_key) if office_key else ''
```

```
342 logger.debug('Rol %s puede crear novedades', sanitizar_log_text(safe_role_key))
```

Propagation path #1

Source config/permissions.py

```
342 logger.debug('Rol %s puede crear novedades', sanitizar_log_text(safe_role_key))
```

```
350 logger.debug('Rol %s tiene acceso al módulo novedades: %s', sanitizar_log_text(safe_role_key), sanitizar_log_text(has_module_access))
```

Propagation path #1

Source config/permissions.py

```
350 | logger.debug('Rol %s tiene acceso al módulo novedades: %s', sanitizar_log_text(safe_role_key), sanitizar_log_text(has_module_access))
```

```
365 | logger.debug('Rol %s puede gestionar novedades', sanitizar_log_text(safe_role_key))
```

Propagation path #1

Source config/permissions.py

```
364 | safe_role_key = sanitizar_log_text(role_key)
```

```
456 | logger.debug('Rol del usuario: %s', sanitizar_log_text(safe_role_key))
```

Propagation path #1

Source config/permissions.py

```
455 | safe_role_key = sanitizar_log_text(role_key)
```

---

8 Sink in file services/auth_service.py

```
33 | logger.info("Intentando validar credenciales de directorio para: %s", sanitizar_log_text(sanitizar_username(username)))
```

Propagation path #1

Source services/auth_service.py

```
33 | logger.info("Intentando validar credenciales de directorio para: %s", sanitizar_log_text(sanitizar_username(username)))
```

```
39 | logger.info("Credenciales de directorio validadas para %s", sanitizar_log_text(sanitizar_username(username)))
```

Propagation path #1

Source services/auth_service.py

```
39 logger.info("Credenciales de directorio validadas para %s", sanitizar_log_text(sanitizar_username(username)))
```

```
44 logger.info("Creando nuevo usuario desde directorio: %s", sanitizar_log_text(sanitizar_username(username)))
```

Propagation path #1

Source services/auth_service.py

```
44 logger.info("Creando nuevo usuario desde directorio: %s", sanitizar_log_text(sanitizar_username(username)))
```

```
47 logger.info("Actualizando usuario existente desde directorio: %s", sanitizar_log_text(sanitizar_username(username)))
```

Propagation path #1

Source services/auth_service.py

```
47 logger.info("Actualizando usuario existente desde directorio: %s", sanitizar_log_text(sanitizar_username(username)))
```

```
58 logger.warning("Falló validación de directorio para %s (id=%s)", sanitizar_log_text(sanitizar_username(username)),
sanitizar_log_text(_new_error_id()))
```

Propagation path #1

Source services/auth_service.py

```
58 logger.warning("Falló validación de directorio para %s (id=%s)", sanitizar_log_text(sanitizar_username(username)),
sanitizar_log_text(_new_error_id()))
```

```
60 logger.info("Intentando validación local para %s", sanitizar_log_text(sanitizar_username(username)))
```

Propagation path #1

Source services/auth_service.py

```
60 logger.info("Intentando validación local para %s", sanitizar_log_text(sanitizar_username(username)))
```

66 logger.info("Validación local exitosa para %s", sanitizar_log_text(sanitizar_username(username)))

Propagation path #1

Source services/auth_service.py

66 logger.info("Validación local exitosa para %s", sanitizar_log_text(sanitizar_username(username)))

78 logger.warning("Credenciales inválidas para %s", sanitizar_log_text(sanitizar_username(username)))

Propagation path #1

Source services/auth_service.py

78 logger.warning("Credenciales inválidas para %s", sanitizar_log_text(sanitizar_username(username)))

---

13 Sink in file utils/ldap_auth.py

105 logger.warning("Usuario válido pero no encontrado en búsqueda: %s", sanitizar_log_text(username))

Propagation path #1

Source utils/ldap_auth.py

105 logger.warning("Usuario válido pero no encontrado en búsqueda: %s", sanitizar_log_text(username))

Propagation path #2

Source services/auth_service.py

36 self.ad_auth.authenticate_user(username, password)

115 logger.warning("Validación de credenciales fallida para usuario: %s", sanitizar_log_text(username))

Propagation path #1

Source utils/ldap_auth.py

```
115 | logger.warning("Validación de credenciales fallida para usuario: %s", sanitizar_log_text(username))
```

Propagation path #2

Source services/auth_service.py

```
36 | self.ad_auth.authenticate_user(username, password)
```

```
120 | logger.error("Error interno validando credenciales para usuario: %s (id=%s)", sanitizar_log_text(username), sanitizar_log_text(_new_error_id()))
```

Propagation path #1

Source utils/ldap_auth.py

```
120 | logger.error("Error interno validando credenciales para usuario: %s (id=%s)", sanitizar_log_text(username), sanitizar_log_text(_new_error_id()))
```

Propagation path #2

Source services/auth_service.py

```
36 | self.ad_auth.authenticate_user(username, password)
```

```
162 | logger.warning("Usuario no encontrado en búsqueda: %s", sanitizar_log_text(username))
```

Propagation path #1

Source utils/ldap_auth.py

```
96 | self._get_user_details(conn, username)
```

Propagation path #2

Source utils/ldap_auth.py

```
162 | logger.warning("Usuario no encontrado en búsqueda: %s", sanitizar_log_text(username))
```

Propagation path #3

Source services/auth_service.py

```
36 self.ad_auth.authenticate_user(username, password)
```

```
80 logger.info("Intentando validar credenciales de directorio para usuario: %s", sanitizar_log_text(username))
```

Propagation path #1

Source utils/ldap_auth.py

```
80 logger.info("Intentando validar credenciales de directorio para usuario: %s", sanitizar_log_text(username))
```

Propagation path #2

Source services/auth_service.py

```
36 self.ad_auth.authenticate_user(username, password)
```

```
93 logger.info("Credenciales validadas correctamente para usuario: %s", sanitizar_log_text(username))
```

Propagation path #1

Source utils/ldap_auth.py

```
93 logger.info("Credenciales validadas correctamente para usuario: %s", sanitizar_log_text(username))
```

Propagation path #2

Source services/auth_service.py

```
36 self.ad_auth.authenticate_user(username, password)
```

---

7 Sink in file utils/permissions.py

```
107 logger.debug('Permisos obtenidos para usuario: role_key=%s, office_key=%s', sanitizar_log_text(safe_role_key), sanitizar_log_text(safe_office_key))
```

Propagation path #1

Source utils/permissions.py

```
105 | safe_role_key = sanitizar_log_text(role_key)
```

```
223 | logger.debug('Filtro de oficina: %s', sanitizar_log_text(safe_office_key))
```

Propagation path #1

Source utils/permissions.py

```
222 | safe_office_key = sanitizar_log_text(office_key) if office_key else ''
```

```
246 | logger.debug('Rol %s puede crear novedades', sanitizar_log_text(safe_role_key))
```

Propagation path #1

Source utils/permissions.py

```
246 | logger.debug('Rol %s puede crear novedades', sanitizar_log_text(safe_role_key))
```

```
250 | logger.debug('Rol %s tiene acceso al módulo novedades: %s', sanitizar_log_text(safe_role_key), sanitizar_log_text(has_module_access))
```

Propagation path #1

Source utils/permissions.py

```
250 | logger.debug('Rol %s tiene acceso al módulo novedades: %s', sanitizar_log_text(safe_role_key), sanitizar_log_text(has_module_access))
```

```
260 | logger.debug('Rol %s puede gestionar novedades', sanitizar_log_text(safe_role_key))
```

Propagation path #1

Source utils/permissions.py

```
259 | safe_role_key = sanitizar_log_text(role_key)
```

```
335 | logger.debug('Rol del usuario: %s', sanitizar_log_text(safe_role_key))
```

Propagation path #1

Source utils/permissions.py

```
334 | safe_role_key = sanitizar_log_text(role_key)
```

```
69 | logger.debug('Rol encontrado por comparación plana: %s', sanitizar_log_text(key))
```

Propagation path #1

Source utils/permissions.py

```
69 | logger.debug('Rol encontrado por comparación plana: %s', sanitizar_log_text(key))
```

FILES	#VULNERABILITY	RULE	CHARACTERISTIC
1	1	Form validation disabled	Security

PRIORITY	CWE	VULNERABILITY TYPE	LANGUAGE
	20	Misconfiguration	html

HTML5 offers a facility for simple (browser-side) input form validation. Input fields could be validated with certain validation attributes (required, pattern...). Both the novalidate attribute in <form> tag, or the formnovalidate attribute in the submit <input> element, deactivate form validation.

The rule reports any HTML5 form with validation disabled.

Please note that client-side validation is not enough to protect the web application against injection attacks, but is a convenient mechanism for end-users to provide valid data.

---

1 Defects in file templates/prestamos/crear.html

```
27 | <form id="prestamoForm" method="POST" novalidate>
```

KIUWAN REPORT

THANK YOU!

contact@kiuwan.com | Partnership:partners@kiuwan.com | +1 6176073353

© Kiuwan Software S.L. – All rights reserved

kiuwan